



UNIVERSIDAD DE
CUNDINAMARCA
Generación Siglo 21

CONEXIÓN BASE DE DATOS
LINEA DE PROFUNDIZACION III

JORGE LUIS PORRAS CEPEDA
Cod 1013635351

WILLIAM ALEXANDER MATA LLANA PORRAS
DOCENTE LINEA DE PROFUNDIZACION III

UNIVERSIDAD DE CUNDINAMARCA EXTENSIÓN CHÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
FACULTAD DE INGENIERÍA
2025



OBJETIVOS

1. Desarrollo de Aplicaciones Web Eficientes
2. Desarrollo de Aplicaciones Más Robustas y Seguras
3. Desarrollo de Aplicaciones Mantenibles
4. Profundización en el Ecosistema Spring
5. Resolución de Problemas Eficaz

Etiquetas usadas

@Getter: Genera automáticamente los métodos "getter" para los campos de la clase. (Obtenedor)

@Setter: Genera automáticamente los métodos "setter" para los campos de la clase. (Establecedor)

@ToString: Genera automáticamente un método toString(), que proporciona una representación de cadena del estado del objeto. (A Cadena)

@EqualsAndHashCode: Genera automáticamente los métodos equals() y hashCode(), que son esenciales para comparar objetos y usarlos en colecciones. (Igualdad y Código Hash)

@AllArgsConstructor: Esta anotación genera automáticamente un constructor que toma un argumento para cada campo de la clase.

Es útil cuando quieres inicializar todos los campos de un objeto al momento de su creación.

@NoArgsConstructor: Esta anotación genera automáticamente un constructor sin argumentos (un constructor "vacío").

Es útil en situaciones donde necesitas crear una instancia de un objeto sin inicializar sus campos inmediatamente.

@Entity: Indica que una clase es una entidad JPA, lo que significa que representa una tabla en la base de datos. (Entidad)

@Id: Especifica la clave primaria de una entidad. (Identificador)

@GeneratedValue: Especifica cómo se deben generar los valores de la clave primaria (por ejemplo, automáticamente por la base de datos). (Valor Generado)

@JoinTable: Especifica la tabla de unión para una relación muchos a muchos. (Tabla de Unión)

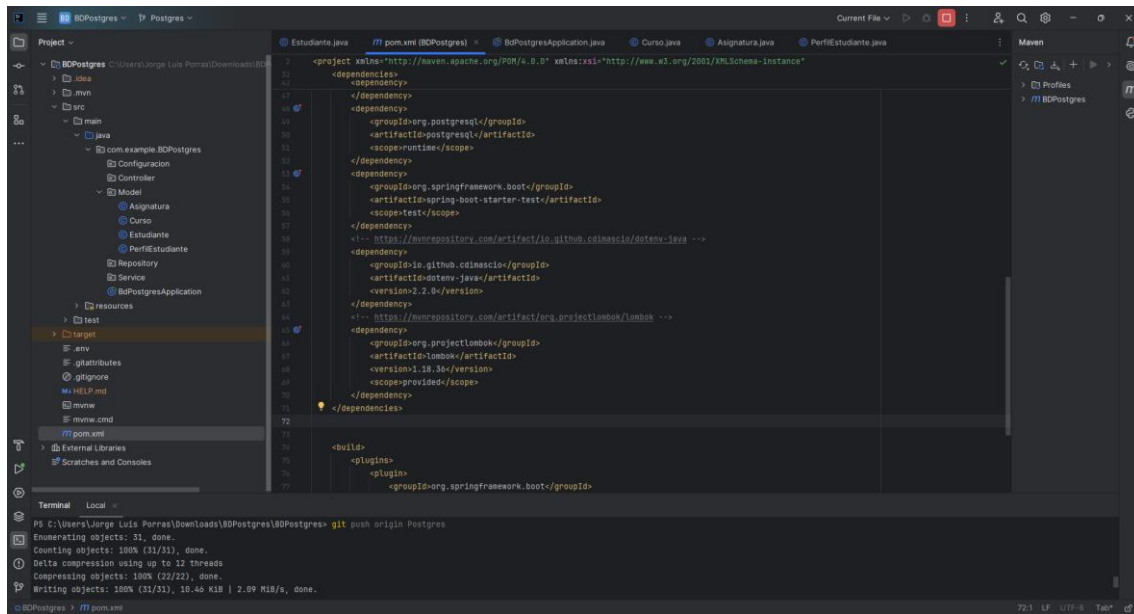
@JoinColumn: Especifica la columna de clave externa en una tabla. (Columna de Unión)

@ManyToMany: Define una relación muchos a muchos entre entidades. (Muchos a Muchos)

@ManyToOne: Define una relación muchos a uno entre entidades. (Muchos a Uno)

@OneToOne: Define una relación uno a uno entre entidades. (Uno a Uno)

Dependencias Usadas



Dotenv Java

La dependencia Dotenv Java tiene la función principal de cargar variables de entorno desde un archivo `.env` en tu aplicación Java. Esto es extremadamente útil para gestionar configuraciones sensibles o específicas del entorno fuera del código fuente de tu aplicación.

Lombok

La dependencia Lombok en Java tiene como función principal reducir significativamente la cantidad de código boilerplate que los desarrolladores deben escribir. En términos sencillos, Lombok automatiza la generación de código repetitivo, lo que permite escribir clases Java más limpias y concisas.

Spring Boot DevTools

La dependencia Spring Boot DevTools es una herramienta diseñada para mejorar la experiencia de desarrollo de aplicaciones Spring Boot. Su función principal es facilitar y acelerar el proceso de desarrollo al proporcionar varias características útiles.

Spring Web

La dependencia Spring Web es un componente fundamental en el ecosistema de Spring Boot, y su función principal es proporcionar las herramientas necesarias para construir aplicaciones web.

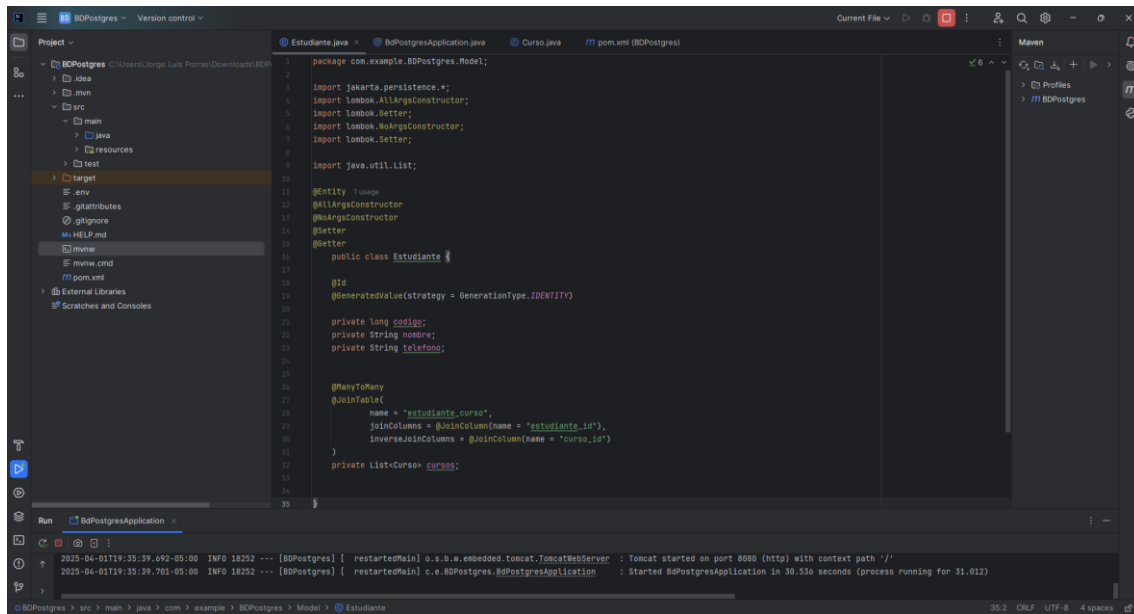
PostgreSQL Driver

La dependencia PostgreSQL Driver, específicamente el controlador JDBC (Java Database Connectivity) para PostgreSQL, tiene la función esencial de permitir que las aplicaciones Java se comuniquen con bases de datos PostgreSQL.

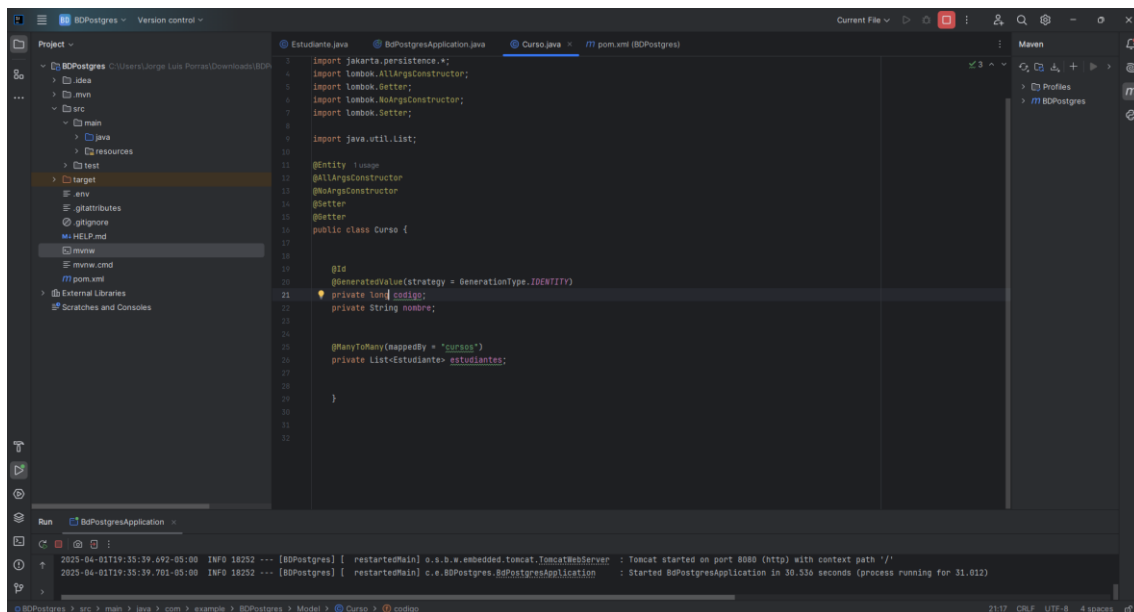
Spring Data JPA

La dependencia Spring Data JPA juega un papel crucial en el desarrollo de aplicaciones Java que interactúan con bases de datos relacionales. Su función principal es simplificar y automatizar el acceso a datos, reduciendo significativamente la cantidad de código repetitivo que los desarrolladores deben escribir.

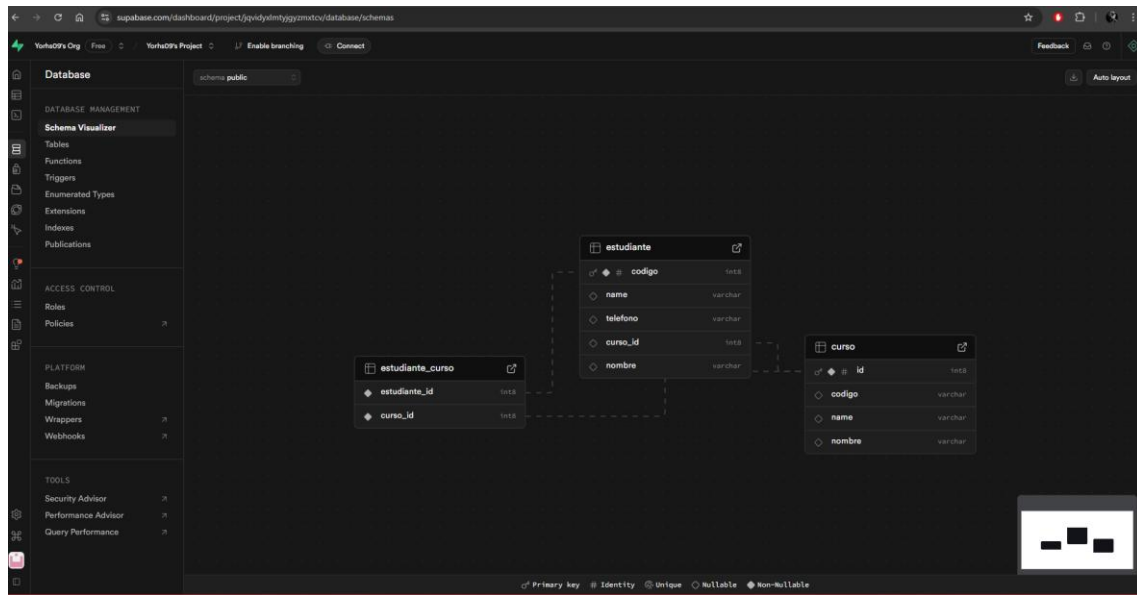
ManyToOne



```
1 package com.example.BDPostgres.Model;
2
3 import jakarta.persistence.*;
4 import lombok.AllArgsConstructor;
5 import lombok.Getter;
6 import lombok.NoArgsConstructor;
7 import lombok.Setter;
8
9 import java.util.List;
10
11 @Entity
12 @AllArgsConstructor
13 @NoArgsConstructor
14 @Getter
15 @Setter
16 public class Estudiante {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private long id;
21
22     private String nombre;
23     private String telefono;
24
25     @ManyToOne
26     @JoinColumn(name = "estudiantes_curso",
27               joinColumns = @JoinColumn(name = "estudiante_id"),
28               inverseJoinColumns = @JoinColumn(name = "curso_id"))
29     private List<Curso> cursos;
30 }
```



```
1 import jakarta.persistence.*;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Getter;
5 import lombok.NoArgsConstructor;
6 import lombok.Setter;
7
8 import java.util.List;
9
10 @Entity
11 @AllArgsConstructor
12 @NoArgsConstructor
13 @Getter
14 @Setter
15 public class Curso {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private long id;
20
21     private String nombre;
22
23     @ManyToMany(mappedBy = "cursos")
24     private List<Estudiante> estudiantes;
25 }
```





ManyToMany

```
10 @Entity
11 @AllArgsConstructor
12 @NoArgsConstructor
13 @Getter
14 @Setter
15 public class Estudiante {
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private long id;
19     private String nombre;
20     private String telefono;
21
22     @ManyToMany
23     @JoinTable(
24         name = "estudiante_asignatura",
25         joinColumns = @JoinColumn(name = "estudiante_id"),
26         inverseJoinColumns = @JoinColumn(name = "asignatura_id")
27     )
28     private List<Asignatura> asignaturas;
29
30     @ManyToMany
31     @JoinTable(
32         name = "estudiante_curso",
33         joinColumns = @JoinColumn(name = "estudiante_id"),
34         inverseJoinColumns = @JoinColumn(name = "curso_id")
35     )
36     private List<Curso> cursos;
37
38     @ManyToOne
39     @JoinColumn(
40         name = "curso_id"
41     )
42     private Curso curso;
43
44     @OneToOne
45     @JoinColumn(
46         name = "perfil_id"
47     )
48     private PerfilEstudiante perfil;
```

```
10 @Entity
11 @AllArgsConstructor
12 @NoArgsConstructor
13 @Getter
14 @Setter
15 public class Asignatura {
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private long id;
19     private String nombre;
20
21     @ManyToMany(mappedBy = "asignaturas")
22     private List<Estudiante> estudiantes;
23
24     @ManyToMany
25     @JoinTable(
26         name = "asignatura_curso",
27         joinColumns = @JoinColumn(name = "asignatura_id"),
28         inverseJoinColumns = @JoinColumn(name = "curso_id")
29     )
30     private List<Curso> cursos;
31 }
32
33 private List<Curso> cursos;
```

Run BdpPostgresApplication

hibernate: alter table if exists estudiante_asignatura add constraint FK59rv04eeagur0r42vnc36knw foreign key (asignatura_id) references asignatura

hibernate: alter table if exists estudiante_asignatura add constraint FK1kxgphny3oajsevd2avdrce foreign key (estudiante_id) references estudiante

2025-04-01T20:38:36.803-05:00 INFO 13040 --- [BdpPostgres] [restartedMain] j.LocalContainerEntityManagerFactoryBean : Initializing JPA EntityManagerFactory for persistence unit 'default'

2025-04-01T20:38:37.608-05:00 INFO 13040 --- [BdpPostgres] [restartedMain] o.s.b.d.e.OptionalLiveReloadServer : LiveReload server is running on port 35729

2025-04-01T20:38:37.651-05:00 INFO 13040 --- [BdpPostgres] [restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'

2025-04-01T20:38:37.661-05:00 INFO 13040 --- [BdpPostgres] [restartedMain] c.w.BdpPostgres.BdpPostgresApplication : Started BdpPostgresApplication in 9.599 seconds (process running for 10.001)



OneToOne

```
package com.example.BDPostgres.Model;

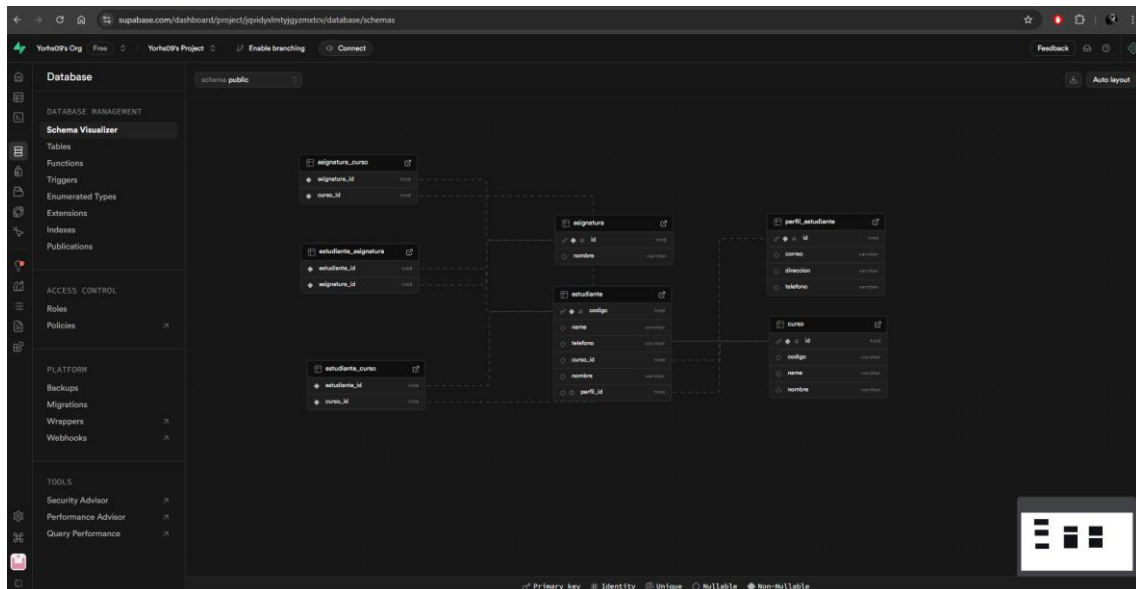
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.ToString;
import lombok.EqualsAndHashCode;

@Entity
@Table(name = "perfil_estudiante")
public class PerfilEstudiante {

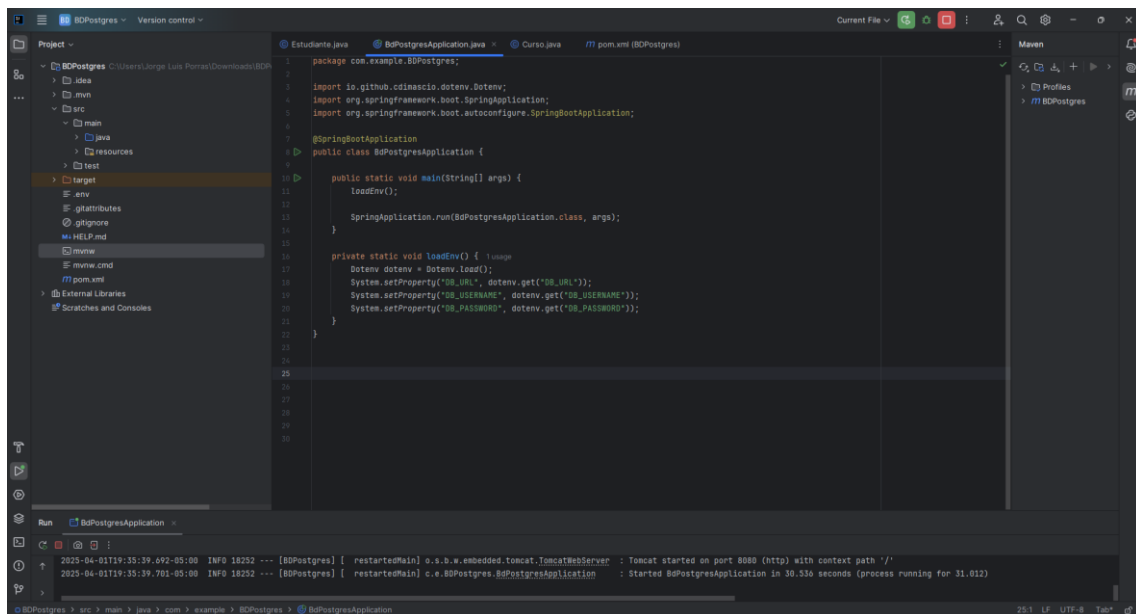
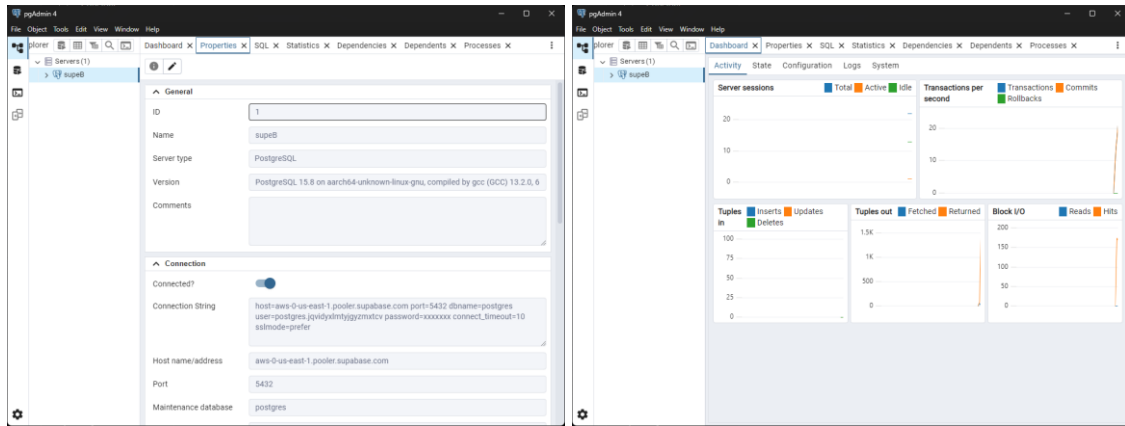
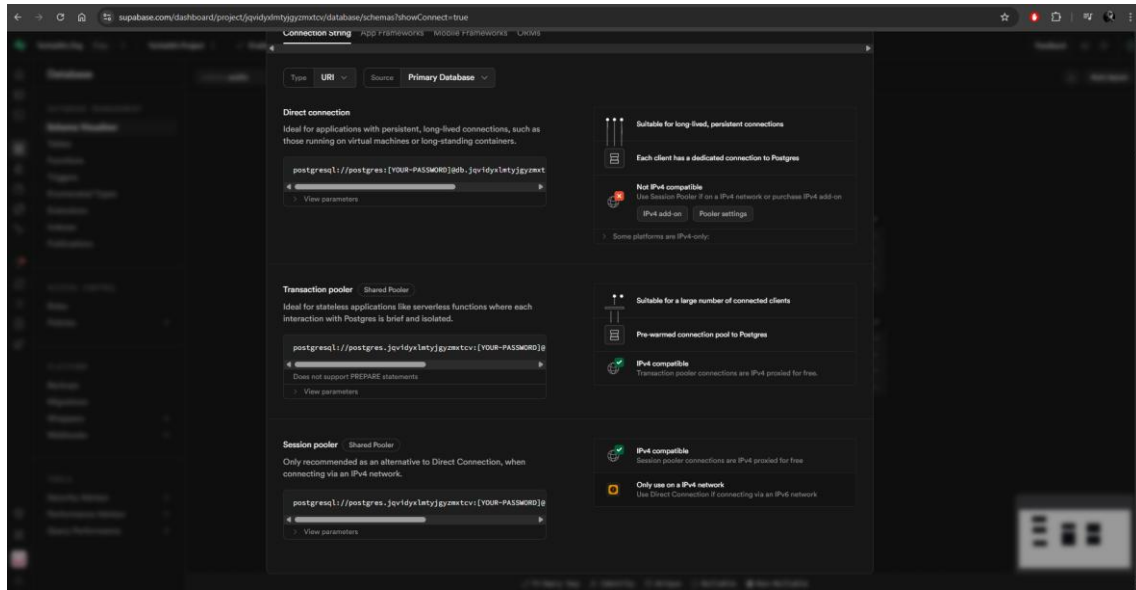
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String direccion;
    private String telefono;
    private String correo;

    @OneToOne(mappedBy = "perfil")
    private Estudiante estudiante;
}
```



Conexión a la base de datos





The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure for 'BDPostgres', including 'src/main/java', 'resources', 'target', and 'pom.xml'.
- Editor:** Displays the 'BDPostgresApplication.java' file with the following code:

```
1 DB_URL=jdbc:postgresql://aws-0-us-east-1.pooler.supabase.com:5432/postgres
2 DB_USERNAME=postgres_jquidylntvjygmxtcv
3 DB_PASSWORD=in4ZK7aR0Tf4s4D
```
- Run Console:** Shows the output of the application run, indicating that Tomcat started on port 8080 and the application started successfully.



Bibliografía

<https://keepcoding.io/blog/sabes-que-es-el-dotenv-en-node-js/>

<https://asjordi.dev/blog/proyecto-lombok-en-java/>

<https://www.arquitecturajava.com/>

<https://gemini.google.com/app?hl=es>

Paginas usadas

<https://start.spring.io/>

<https://mvnrepository.com/>

<https://www.postgresql.org/ftp/pgadmin/pgadmin4/v9.1/windows/>

<https://supabase.com/>